

ROB 521 H Mobile Robotics and Perception

Deliverables to Simulation (Task 3: Construct an Occupancy Grid Map)

- (1) The video of the RVIZ window during mapping of the environment is attached.

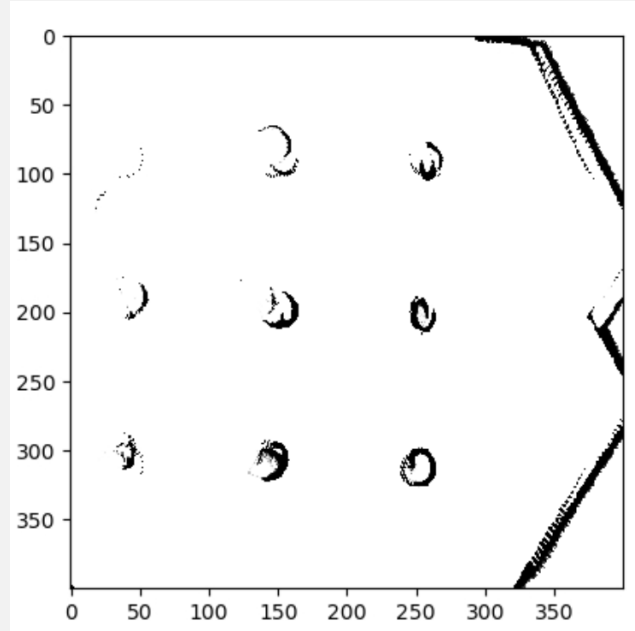


Figure 1: The mapped environment.

- (2) A brief explanation of our code with respect to occupancy map construction is as follows:

1. For the `scan_cb()` function:

- (a) It gets current odometry robot pose and transform it to the frame of map;
- (b) It reads and process `scan_msg`. For all rays in a scan:
 - i. They are downsampled by a constant `SCAN_DOWNSAMPLE = 1` (meaning that no down-sampling is performed);
 - ii. For a single ray in selected rays after downsampling, it is first filtered by a valid interval of range measurement `[scan_msg.range_min, scan_msg.range_max]`, then the transformed odometry robot pose in map frame as well as the range measurement of this ray is discretized according to the `CELL_SIZE = 0.01` of the grid, finally these discretized `x_start`, `y_start`, `theta` and `range_mes` are used in `ray_trace_update()` to update the log odds `self.log_odds` and the occupancy map `self.np_map`.
- (c) The updated occupancy map is published.

2. For the `ray_trace_update()` function (key in `scan_cb()` that updates the occupancy map):

- (a) According to the `x_start`, `y_start`, `theta` and `range_mes`, locate the pixels in the grid map (r_0, c_0) , (r_1, c_1) , (r_2, c_2) that are respectively the start of the laser, the collision point of the laser at the obstacle, and the extended end of the occupied area that is control by a constant `NUM_PTS_OBSTACLE = 3`;

- (b) The empty pixels and occupied pixels along the laser are determined by respectively setting the start point and end point as (r_0, c_0) , (r_1, c_1) and (r_1, c_1) , (r_2, c_2) in `ray_trace()`;
- (c) These pixels are filtered to be in the legal range of the grid map;
- (d) The log odds at empty and occupied pixels are updated by $\alpha = \beta = 1$ and the occupancy probability map is updated accordingly.

(3) According to the lab document, simulated odometry measurements have some noise augmented to them. Aside from the noise of sensors, the adjustment of the parameter `NUM_PTS_OBSTACLE` could be a potential source of error since overly large setting could erroneously largen the obstacle. Drift in localization caused by dead reckoning from noisy sensor readings will also be a problem in the algorithm, especially during fast robot movement. This is because higher speeds lead to greater integration errors in odometry over shorter time intervals. Localization and thus mapping gets worse the longer the robot travels because the effects of noise of sensors accumulates, causing our map to drift.

Deliverables to Experiments (Task 4: Calibrate the TurtleBot)

(1) For the wheel radius calibration experiment, the robot was driven in a straight line for 1 meters and the wheel radius was calculated to be 0.0326 meters, close to the given wheel radius of 0.03 meters. The wheel rotated about 4.88 times in the 1 meter distance travelled. The robot itself did not rotate, it was driven in straight line.

To determine wheel seperation, the robot was rotated on the spot 3 times (1080 degrees), without moving forward. We determined a wheel seperation of 0.303 meters compared to the given wheel seperation of 0.287 meters. The robot was driven for around 2.86 meters during its rotation.

(2) The code was completed by completing the functions `l3_estimate_wheel_radius.py` and `l3_estimate_wheel_baseline.py`. We simply plugged in the calibration formulas from lecture on wheel odometry and dead reckoning.

The formula used for wheel radius is:

$$r = \frac{\text{DRIVEN_DISTANCE} \times \text{TICKS_PER_ROTATION}}{\pi(\Delta\text{encoder}_{\text{left}} + \Delta\text{encoder}_{\text{right}})}$$

where $\Delta\text{encoder}_{\text{left}}$ is from the left wheel encoder counting the number of ticks since the beginning of the experiment, and $\Delta\text{encoder}_{\text{right}}$ is defined similarly from the right wheel encoder.

The formula used for wheel seperation is:

$$\text{separation} = \frac{\text{WHEEL_RADIUS} \times |\Delta\text{encoder}_{\text{right}} - \Delta\text{encoder}_{\text{left}}|}{\pi \times \text{NUM_ROTATIONS} \times \text{TICKS_PER_ROTATION}}$$

where $\Delta\text{encoder}_{\text{right}}$ and $\Delta\text{encoder}_{\text{left}}$ are defined the same as in wheel radius calibration, and `NUM.ROTATIONS` is 3; the number of times we rotated the robot.

(3) Wheel radius: 0.0326 meters, Wheel Baseline: 0.287 meters, each rounded to 3 significant digits.

(4) For both wheel radius and baseline experiments, trials were conducted 3 times and the mean computed to reduce uncertainty. Overall, our results are quite close to what was given from data sheet. Some possible source of uncertainty includes wheels slipping where the the robot would take more rotation to traverse a set distance. Other possible errors could include measuring uncertainty where the robot may not traverse on the exact path the ruler measures. The robot could travel slightly to the left or slightly to the right due to power difference between the motors. Sometimes we also experienced problems where after pressing stop on the tele-operation control, the robot would takes a few seconds before stopping. We tried mitigates the uncertainties by choose a relatively clean floor surface, ensuring that the robot battery is fully charged and ran the calibration several times for best results.

Deliverables to Experiments (Task 5: Motion Estimation)

(1) The two plots from the two experiments comparing our odom estimates with the onboard one are shown below.

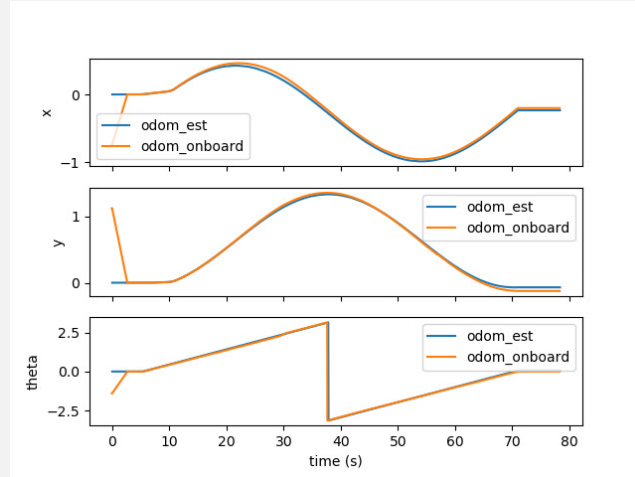


Figure 2: Estimation of simple circular movement.

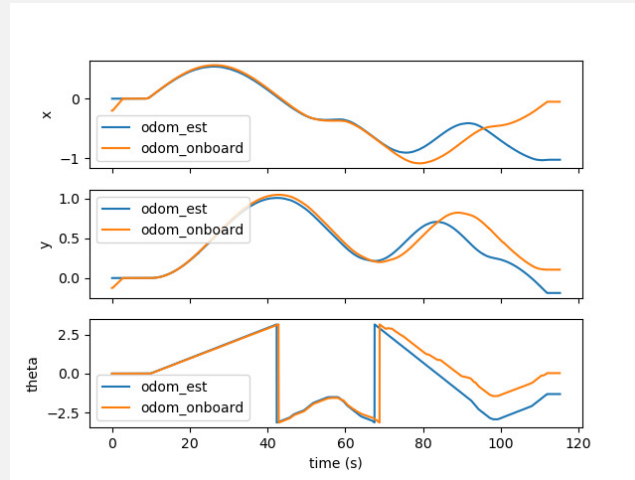


Figure 3: Estimation of following a more complex path - drawing the number 8

(2) Initially, the estimated odometry aligns well with the onboard odometry, but as the TurtleBot continues to move, discrepancies begin to grow due to cumulative error. A likely contributor to this drift is wheel slippage. This can occur when the TurtleBot traverses uneven terrain or surfaces made of different materials—such as paper tape or tile seams—causing the wheels to rotate without producing the expected movement. Since both onboard and estimated odometry rely on encoder readings, slippage introduces error into both. However, the onboard odometry incorporates IMU corrections, which help compensate for some of the slip-induced error, leading to divergence between the two over time. Note also that our odometry estimate is based on our own wheel radius and baseline estimates; so error carries over in the odometry calculations and can result in deviations from onboard estimates.

The estimates are close to actual paths, however there are still gaps, and the gap of the complex task is larger than the simple one. A possible source of error would be the possible wheel slip and the noisy data from the onboard measurement units that accumulates as the robot traverse the path. Fast robot movements seems to also impact onboard odometry estimations. Slower robot movements seems to contribute to best odometry. In the simple path, we drove the robot with a radius a bit less

than 1 meter, so not much time for error accumulation. we also drove slowly. This is why Figure 2 results are better than Figure 3.

Deliverables to Experiments (Task 6: Mapping)

- (1) A live mapping demo has been examined by the TA during the lab session.

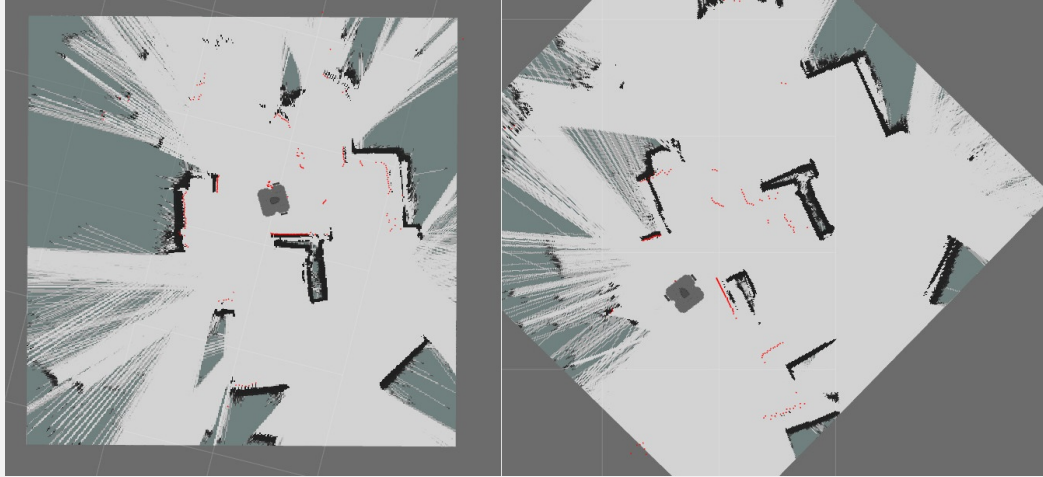


Figure 4: Estimation of simple circular movement.

- (2) A source of error not present in simulation but significant in real-world experiments is the imperfect physical environment. For example, uneven flooring or slight imperfections in the robot's tires can lead to inconsistent wheel contact with the ground, causing slippage or irregular movement that directly impacts wheel encoder readings. Since odometry assumes consistent, slip-free motion, these irregularities introduce drift in localization and can degrade the accuracy of environment mapping. Additionally, the presence of other robots in the testing space introduces dynamic obstacles that can be misinterpreted as part of the static environment, resulting in incorrect or noisy map features. Movement of other groups and their robots during setup further disrupts sensor readings, particularly for systems relying on laser scanners or depth sensors, as these temporary obstacles may be falsely integrated into the map or occlude important features. Together, these real-world factors contribute to errors in both localization and mapping that are not accounted for in idealized simulations.